# Neural Network Architectures For The Forward Kinematics Problem in Robotics[1]

*L. Nguyen, R.V. Patel, K. Khorasani*

Department of Electrical and Computer Engineering
Concordia University, Montreal, Quebec
CANADA H3G 1M8

## ABSTRACT

**In this paper, various neural network models are considered for solving the robot forward kinematics problem. It is found that certain models with proper training strategies can generate a fairly accurate solution for the robot forward kinematics problem.**

## 1 Introduction

Manipulator kinematics is a study of the geometry of manipulator arm motions. Since the performance of specific manipulator tasks is achieved through the movement of the manipulator linkages, kinematics is of fundamental importance in robot design and control

A kinematic equation provides the relationship between the joint displacement and the resulting end-effector position and orientation. The problem of finding the end-effector position and orientation for a given set of joint displacements is referred to as the forward kinematics problem. That is, the forward kinematics problem allows one to specify in a unique manner the relationship between the (n x 1) joint vector $\theta$ and the (m x 1) Cartesian vector X as:

$$X = F(\theta)$$

where F is the function defining the forward kinematic relation of the manipulator. Normally, the forward kinematic equation can be obtained from the spatial geometry of the manipulator or by solving certain matrix algebraic equations. As the number of degrees of freedom (n) increases, the kinematic equation becomes more complex. Hence, the amount of computation required to compute the end-effector position can become quite large.

Neural networks provide a fast method of learning to produce a set of output states given a set of input states. The advantages of using a neural network approach in robotics applications such as in the forward kinematics problem are:

- no programming is required;
- the time to obtain a solution is independent of the number of manipulator degrees of freedom;
- fault tolerance;
- capability of operating in real-time.

In this paper, various neural network architectures are used to solve the forward kinematics problem in robotics. Its purpose is to identify the advantages and disadvantages of each architecture in this robotic application.

## 2 Neural Network Architectures

Recently, there has been a lot of interest in using neural networks to solve various problems in robotics like inverse dynamics and inverse kinematics. Kawato et al [1] has proposed a general architecture to solve such problems. However, their approach requires knowledge of the system characteristics as well as the system input-output relationship.

[1]This research was supported by the Natural Sciences and Engineering Council of Canada



Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 28,2024 at 15:24:18 UTC from IEEE Xplore. Restrictions apply.

In this paper, a general solution to the forward kinematics problem is offered. No knowledge of the manipulator structure is assumed.A multilayer feedforward configuration is designed to model the manipulator kinematics. This term must convert any joint space trajectory into Cartesian space trajectory. Under our consideration, a manipulator is treated as a black box. The network must be taught to learn the forward kinematic relationship solely by observation of the manipulator's inputs and outputs.

## 2.1 The Manipulator Architecture

We consider a rigid-link, open-chain two degrees-of-freedom planar arm. To make the arm move to the desired end point (X,Y) , the required joint angles $(\theta_1, \theta_2)$ are generated by the motors at the joints.

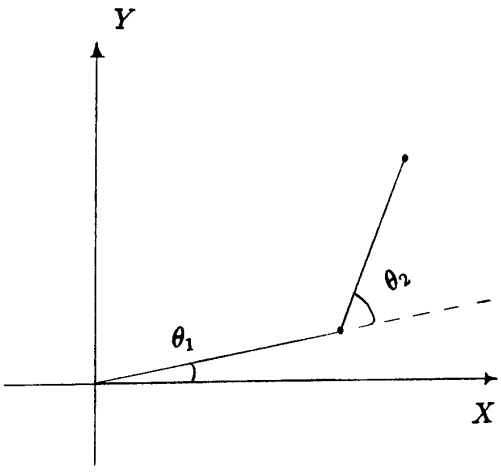


Figure 1. Two link planar arm.

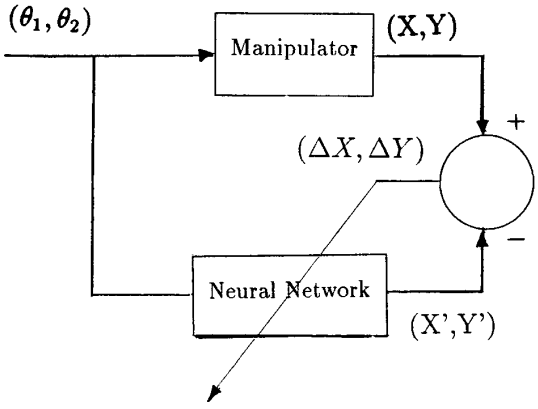


Figure 2. The Supervized Learning.

The forward transformation to map joint coordinates $(\theta_1, \theta_2)$ to the end effector position (X,Y) is:

$$X = l_1 cos(\theta_1) + l_2 cos(\theta_1 + \theta_2)$$
$$Y = l_1 sin(\theta_1) + l_2 sin(\theta_1 + \theta_2)$$

where:

$\theta_1$ , $\theta_2$ = joint angles of the first and second link
$l_1$ , $l_2$ = length of first and second link.

## 2.2 Network Training Strategy

Since the system characteristics are unknown, the supervised learning method is used as in Figure 2. The drawback of this learning is that it is off line procedure. Once the network is trained and installed in the system, no more training is allowed. Therefore, the set of training data must be uniformly distributed over the region of interest so that the network can interpolate the intermediate points [2].

The set of training samples are collected from measurements. This is done by setting the planar arm at several desired inputs (joints angles) and measuring the corresponding outputs (end-effector position in Cartesian space). To simplify the problem, only a quarter of the robot workspace is trained. The training value of joint angles are uniformly distributed from 0 to 1.57 radians. In the following sections, various architectures are employed to implement the above learning scheme.

## 2.3 The Back Propagation Network

According to Hecht-Nielsen [3], for any function F and any $\epsilon > 0$, there exists a three-layer back-propagation network that can approximate F to within $\epsilon$ mean-squared error accuracy. Based on this result, we have construct a three-layer network to model the robot forward kinematics. The first and second hidden layers have ten nodes with a sigmoid transfer function for each node. The last layer has a



Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 28,2024 at 15:24:18 UTC from IEEE Xplore. Restrictions apply.

number of nodes equal to the number of outputs. Since the output is greater than one, each node of the output layer performs the summation function instead of the sigmoid function. The learning procedure follows the generalized delta rule.To speed up the convergence rate, we set a high learning rate in the first hidden layer and a low learning rate in the next two layers [4]. In this case, we use learning rates of 0.4 and 0.1 respectively.

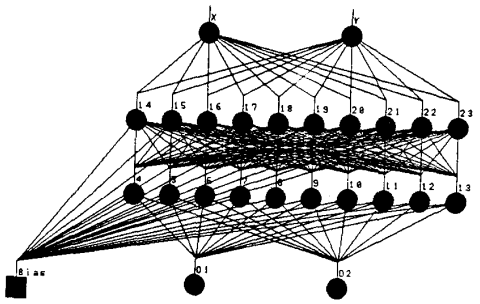

Figure 3. The Back-propagation Network.

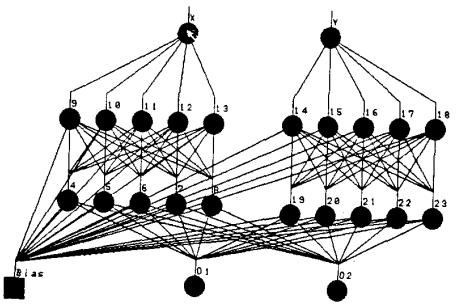

Figure 4. The Back-propagation Network with Output Splitting.

## 2.4 The Back-Propagation Network with Output Splitting

This is a modified version of the previous back-propagation network. Here, there is a separate network for each output. Each network is a three layer network. There are five nodes in each of the first and second hidden layers. Nodes of the first and second layer perform the sigmoid function. In the last layer, each node performs a linear summation. The learning procedure is also the generalized delta rule. The learning rate of the hidden layers are the same as those in the previous back-propagation network.

## 2.5 The Functional Link Network

This is a single layer network whose inputs are the linearly separate components of the kinematic equations. The network is composed of eight nodes. The inputs of these nodes are: $\theta_1, \theta_2, sin(\theta_1)$, $sin(\theta_2), cos(\theta_1), cos(\theta_2)$, $sin(\theta_1 + \theta_2)$ and $cos(\theta_1 + \theta_2)$. The learning procedure of the functional link network is the generalized delta rule. The learning rate used is 0.4.

## 2.6 The Counter Propagation Network

The motivation for using a counter-propagation network to solve the forward kinematics problem is to employ its characteristics in function approximation and in "rapid prototyping"[5]. The counter propagation network used to solve the forward kinematics problem is showed in Figure 6. The network is composed of three layers. The first hidden layer is the normalizing layer that normalizes the input vector into a unit sphere. The second layer is the Kohonen layer that acts as a nearest neighbor classifier. The Kohonen layer measures the cosine of the angle between the input vector X and the weight vector W. The node with the closest weight vector wins the competition. The learning procedure of this layer is the Kohonen learning rule. The output layer is the Grossberg Outstar layer which essentially learns to produce



Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 28,2024 at 15:24:18 UTC from IEEE Xplore. Restrictions apply.

a certain output when a particular input is applied. The output layer follows the Widrow-Hoff learning rule. This counter-propagation network works as a lookup table where the closest input is found and its equivalent output is read. Then, the number of nodes in the Kohonen layer determines how accurately we model the robot forward kinematics. In this particular problem, the number of nodes in the Kohonen layer is thirty two which is about one half of the number of training samples.

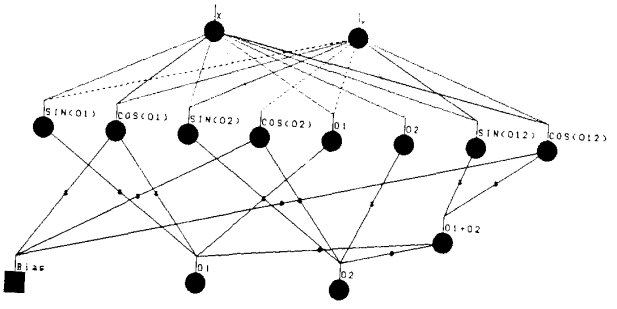


Figure 5. The Functional Link Network.

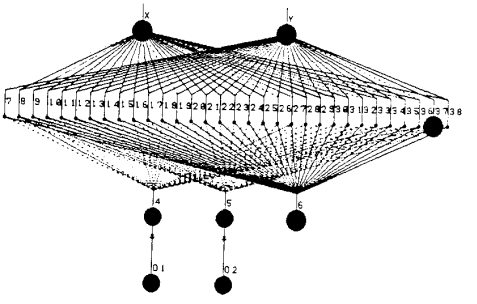


Figure 6. The Counter-Propagation Network.

## 3 Simulation Results

All the results have been obtained by simulation on a SUN 3/60 work station. The networks were trained with a set of sixty four desired inputs/outputs collected from measurements. All the weights of the networks were randomly initialized from -0.5 to +0.5. In the output layer of each network, all the weights were initialized to zero. This is to avoid the case in which a local minima predominates when the training process is started. The training process was stopped when the average error was under 10 per cent.

To test, the performance of the network, the following trajectory was used:

$$\theta_1 = 0.003 * t$$
$$\theta_2 = 0.003 * t$$

where t ranges from 0 to 500. The following figure illustrates the desired trajectory.

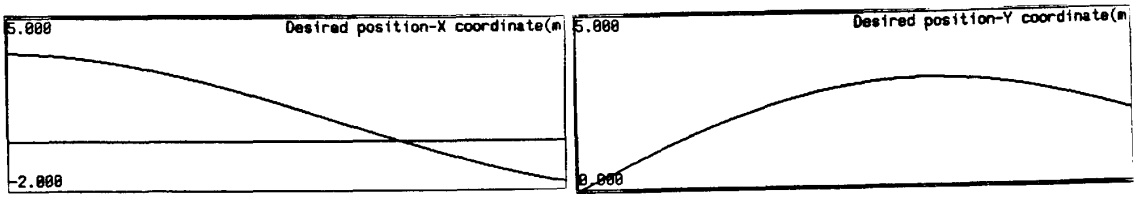


Figure 7. The Desired Trajectory.



Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 28,2024 at 15:24:18 UTC from IEEE Xplore. Restrictions apply.

Figure 8-11 show the simulation results obtained after training the four different networks considered in this paper. The error in each case is measured as the distance beween the desired and the actual Cartesian positions of the end-effector, and is given in centimeters.

# 4 Discussion

The simulation results show that the functional link network converges quickly to the desired trajectory. It provides the best solution of all the networks considered in this paper. The drawback of this approach is that the manipulator kinematic must be known a priori. If a different structure is used, then the network must be redesigned. In contrast, the other three considered networks do not assume any knowledge of the manipulator kinematic structure. The corresponding solutions are consequently less accurate and the networks in general requires more training iterations. The back-propagation network gave a reasonably accurate solution However, the network has a high degree of complexity. The back-propagation network with output splitting has less network complexity but required more training iterations to achieve similar accuracy as the back-propagation network without output splitting. However, the reduced complexity of the network resulted in an overall shorter training time. The counter-propagation network essentially performs piece-wise linear approximation of the trajectory. It had a faster convergence rate compared to the back-propagation networks but gave the largest error. To achieve a better estimation of the output, the Kohonen layer of the network can be modified so that two nodes with the closest weight vector to the input vector win the competition. Figure 12 shows the simulation results of this modified counter-propagation network.

# 5 Conclusions

It has been demonstrated in this paper that a three-layer back-propagation network is capable of learning the forward kinematics of a rigid-link, open-chain manipulator without any knowledge of the manipulator's kinematic structure. The simulation results have shown that by properly training such a network, it is possible to model the forward kinematics with an acceptable degree of accuracy. However, our results have also indicated that if information about the kinematic structure of a manipulator is available, a functional link network gives, by far, the most accurate results. Future work will focus on criteria for selecting the number of nodes in hidden layers and the learning rate.

REFERENCES

[1] M. Kawato et al, 'A Hierarchical Neural Network Model for Control and Learning of Voluntary Movement with Application to Robotics', IEEE Control Systems Magazine, pp 8-15, April 1988.

[2] D. Psaltis, A. Sideris, and A. Yamamura, 'A Multilayered Neural Network Controller', IEEE Control System Magazine, pp 17-21, April 1988.

[3] R. Hecht-Nielsen, 'Theory of The Back Propagation Network', Proc. Int. Joint Conference on Neural Networks, Washington D. C., 1989.

[4] V. Zeman, R. V. Patel and K. Khorasani, 'A Neural Network Based Control Strategy for Flexible-Joint Manipulators', IEEE conference on Decision and Control, pp. 1759, Tampa, December 1989

[5] R. Hecht-Nielsen, 'Applications of Counterpropagation Networks', Neural Networks, Vol. 1, pp 131-139, 1988.

Table 1. Comparion of Neural Network Architectures

| Network Architecture | number of outputs (m) | number of hidden layers | number of nodes per hidden layer(n) | Training Time Complexity | Number of Training iterations |
|---|---|---|---|---|---|
| Back-propagation | 3 | 2 | 10 | $O(n^2)$ | 200 |
| Back-propagation Output Splitting | 3 | 2 | 5 | $O((\frac{n}{m})^2)$ | 300 |
| Functional Link | 3 | 1 | 8 | $O(n)$ | 20 |
| Counter-propagaton | 3 | 1 | 32 | $O(n)$ | 50 |



Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 28,2024 at 15:24:18 UTC from IEEE Xplore. Restrictions apply.

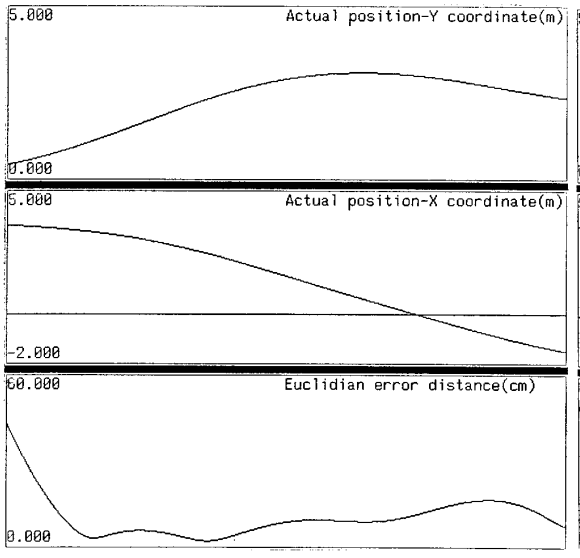


Figure 8. Simulation results for the Back-propagation Network after 200 iterations

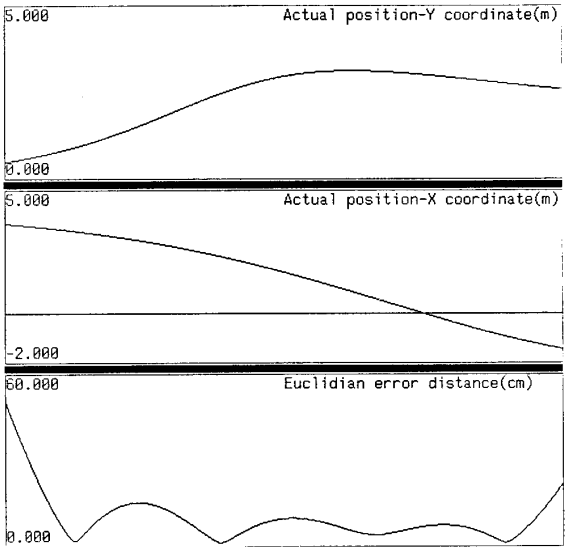


Figure 9. Simulation results for the Back-propagation Network with Output Splitting after 300 iterations.

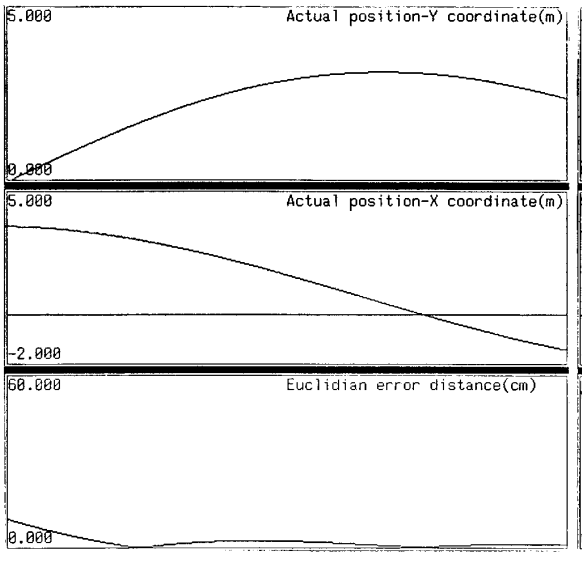


Figure 10. Simulation results for the Functional Link Network after 20 iterations.

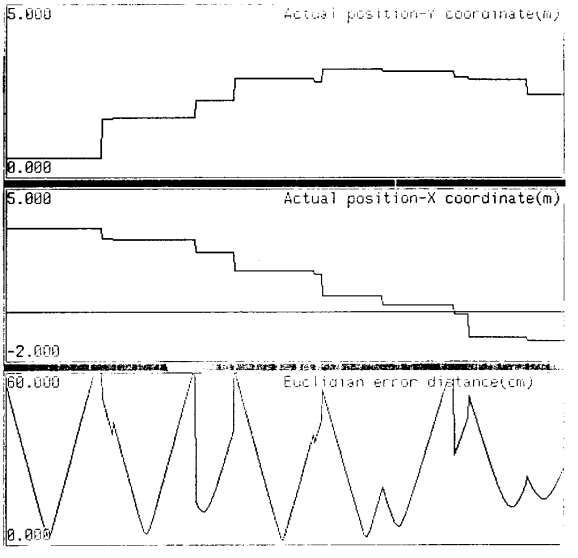


Figure 11. Simulation results for the Counter-propagation Network after 50 iterations.



Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 28,2024 at 15:24:18 UTC from IEEE Xplore. Restrictions apply.

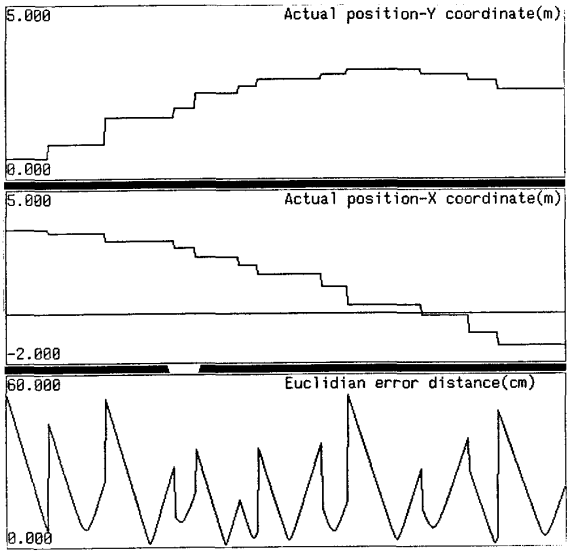


Figure 12. Simulation results for the Modified Counter-propagation Network after 50 iterations.



Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on May 28,2024 at 15:24:18 UTC from IEEE Xplore. Restrictions apply.